

Unitree Week 6

Presentation by Mario Ortiz

Recap

Installed ROS2 and Gazebo on Ubuntu. Implemented g1pilot (the open sourced package for using G1 Unitree robot).

- Linux commands for opening g1pilot:
`source /opt/ros/jazzy/setup.bash`
`source ~/g1_ws/install/setup.bash`
`ros2 launch g1pilot g1_visualization_only.launch.py`

Error running “ros2 launch g1pilot bringup_launcher.launch.py”

```
ros2 launch g1pilot bringup_launcher.launch.py
```

```
[INFO] [launch]: All log files can be found below
```

```
/home/mario/.ros/log/2025-12-16-14-59-09-824957-mario-Inspiron-15-3511-5502
```

```
[INFO] [launch]: Default logging verbosity is set to INFO
```

```
[ERROR] [launch]: Caught exception in launch (see debug for  
traceback): "package 'livox_ros_driver2' not found, searching:  
['/home/mario/g1_ws/install/g1pilot', '/opt/ros/jazzy']"
```

Conclusion from Errors:

- From the error obtained from running “ros2 launch g1pilot bringup_launcher.launch.py”, it was realized that several outside dependencies are needed to run these programs from g1pilot (other than the visualization setup).
 - In regards to bringup, it is necessary to contain Starts drivers for **Livox LiDAR Sensors**.
- Other files require the bringup file to work.
- mola_launcher.launch.py requires a controller for G1.
- In conclusion, these files all require the necessity of a physical G1 robot to function. Only the visualization launcher is meant to work on the machine.

g1_visualization_only.launch.py

```
home > mario > g1_ws > src > g1pilot > launch > g1_visualization_only.launch.py
1  #!/usr/bin/env python3
2  """
3  Simple G1 visualization launch file for RViz only (no real robot needed)
4  """
5
6  from launch import LaunchDescription
7  from launch.actions import DeclareLaunchArgument
8  from launch.substitutions import LaunchConfiguration
9  from launch_ros.actions import Node
10 from launch_ros.parameter_descriptions import ParameterValue
11 from ament_index_python.packages import get_package_share_directory
12 import os
13
14 def generate_launch_description():
15     # Get package directory
16     pkg_share = get_package_share_directory('g1pilot')
17
18     # URDF file
19     urdf_file = os.path.join(pkg_share, 'description_files/urdf/29dof.urdf')
20
21     # RViz config file
22     rviz_config_file = os.path.join(pkg_share, 'config/29dof.rviz')
23
24     # Read URDF
25     with open(urdf_file, 'r') as infp:
26         robot_desc = infp.read()
27
28     # Launch arguments
29     use_sim_time = LaunchConfiguration('use_sim_time')
30
31     # Robot state publisher
32     robot_state_publisher = Node(
33         package='robot_state_publisher',
34         executable='robot_state_publisher',
35         name='robot_state_publisher',
36         output='screen',
37         parameters=[
38             'robot_description': robot_desc,
39             'use_sim_time': use_sim_time
40         ])
41
42     # Joint state publisher (for manual control in RViz)
43     joint_state_publisher_gui = Node(
44         package='joint_state_publisher_gui',
45         executable='joint_state_publisher_gui',
46         name='joint_state_publisher_gui',
47         output='screen')
48
49     # RViz
50     rviz = Node(
51         package='rviz2',
52         executable='rviz2',
53         name='rviz2',
54         output='screen',
55         arguments=['-d', rviz_config_file] if os.path.exists(rviz_config_file) else [])
56
57     return LaunchDescription([
58         DeclareLaunchArgument(
59             'use_sim_time',
60             default_value='false',
61             description='Use simulation time'
62         ),
63         robot_state_publisher,
64         joint_state_publisher_gui,
65         rviz,
66     ])
67
68
69
70
71
```

Code explanation:

- `pkg_share`: locates URDF files and RViz config files from the `g1pilot` directory.
- `urdf_file`: loads the static URDF file (no controllers, etc.)
- `use_sim_time`: runs RViz
- `robot_state_publisher`: reads URDF, publishes “TF” transforms for every joint, updates RViz every time joints are changed.
- `rviz` function: opens RViz based on configuration file.

teleoperation_launcher.launch.py

```
home > mario > g1_ws > src > g1pilot > launch > tele ~/g1_ws/src/g1pilot/launch/teleoperation_launcher.launch.py
1  from launch import LaunchDescription
2  from launch_ros.actions import Node
3
4  def generate_launch_description():
5      return LaunchDescription([
6          Node(
7              package='g1pilot',
8              executable='joystick',
9              name='joystick',
10             output='screen',
11             parameters=[
12                 {'joystick_name': 'Pro Controller',
13                 }],
14         ),
15
16         Node(
17             package='g1pilot',
18             executable='joy_mux',
19             name='joy_mux',
20             output='screen'
21         ),
22
23         Node(
24             package='g1pilot',
25             executable='ui_interface',
26             name='ui_interface',
27             output='screen'
28         ),
29
30     ])
```

Code explanation:

- Contains custom executable within g1pilot package.
 - 'joystick' signifies that a physical controller is needed to utilize this launch file.
 - joy_mux:
 - ui_interface:

Next steps:

- What “g1_visualization_only.launch.py” does is provide the ability to view the G1 on our laptop, test the limitations of the joints, and become accustomed to the URDF structure of the robot.
- Using the structure of the URDF files, we can create our own launch.py that allows us to simulate whatever we want.
- In our case, we may want to start with moving the robot with a controller.

Main Components of a Unitree Simulation

- URDF model (or XACRO)
- Meshes
- ros2_control hardware interface
- Gazebo plugin (Harmonic in our case)
- Controllers (YAML file)

Unified Robot Description Format (URDF)

URDF files are structured XML files utilized for describing the Unitree robot's physical structure for simulation/visualization purposes. The URDF file will contain:

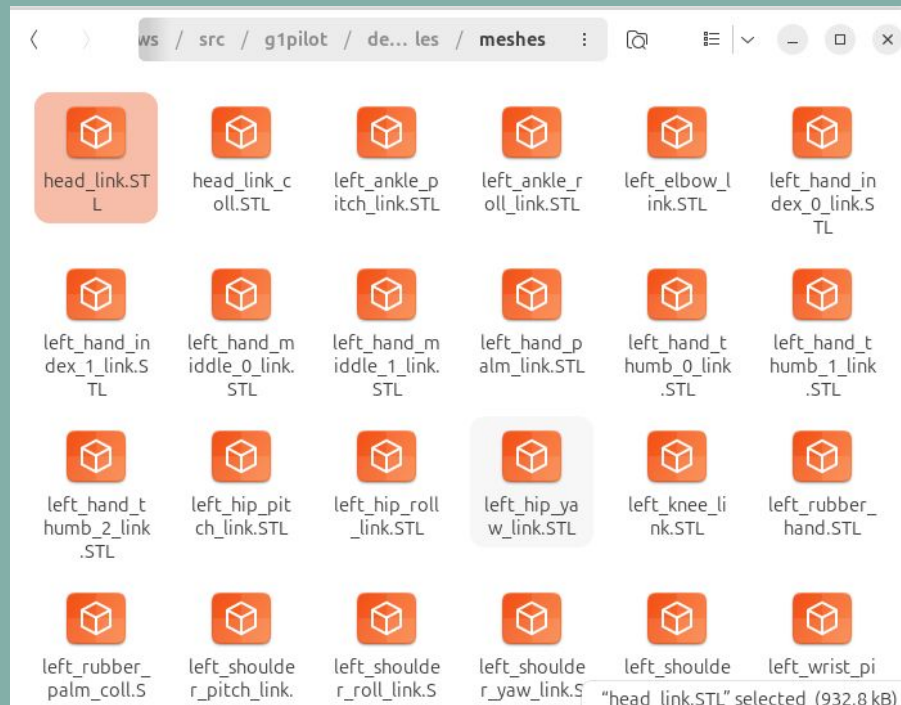
- Links representing rigid body parts of the robot.
 - These links include visual geometry, collision geometry, and inertial properties.
- Joints connecting the links and defining motion
- Transforms, for defining how each link is positioned

Overall URDF is essential for simulating the physics of the robot.

```
home > mario > gl_ws > src > glpilot > description_files > urdf > 29dof.urdf
1  <robot name="gl_29dof_rev_1_0">
2    <mujoco>
3      <compiler meshdir="meshes" discardvisual="false"/>
4    </mujoco>
5
6    <!-- [CAUTION] uncomment when convert to mujoco -->
7
8    <link name="pelvis">
9      <inertial>
10        <origin xyz="0 0 -0.07605" rpy="0 0 0"/>
11        <mass value="3.813"/>
12        <inertia ixx="0.010549" ixy="0" ixz="2.1E-06" iyy="0.0093089" iyz="0" izz="0.0079184"/>
13      </inertial>
14      <visual>
15        <origin xyz="0 0 0" rpy="0 0 0"/>
16        <geometry>
17          <mesh filename="package://glpilot/description_files/meshes/pelvis.STL"/>
18        </geometry>
19        <material name="dark">
20          <color rgba="0.2 0.2 0.2 1"/>
21        </material>
22      </visual>
23    </link>
24    <link name="pelvis_contour_link">
25      <inertial>
26        <origin xyz="0 0 0" rpy="0 0 0"/>
27        <mass value="0.001"/>
28        <inertia ixx="1e-7" ixy="0" ixz="0" iyy="1e-7" iyz="0" izz="1e-7"/>
29      </inertial>
30      <visual>
31        <origin xyz="0 0 0" rpy="0 0 0"/>
32        <geometry>
33          <mesh filename="package://glpilot/description_files/meshes/pelvis_contour_link.STL"/>
34        </geometry>
35        <material name="white">
36          <color rgba="0.7 0.7 0.7 1"/>
37        </material>
38      </visual>
39      <collision>
40        <origin xyz="0 0 0" rpy="0 0 0"/>
41        <geometry>
42          <mesh filename="package://glpilot/description_files/meshes/pelvis_contour_link.STL"/>
43        </geometry>
44      </collision>
45    </link>
46    <joint name="pelvis_contour_joint" type="fixed">
47      <parent link="pelvis"/>
48      <child link="pelvis_contour_link"/>
49    </joint>
50  </robot>
```

Meshes

Meshes are 3D models used for defining the robot's shape, appearance, and collision geometry. They work with the URDF in the visualization and physics of the robot, ensuring the robot looks like the real Unitree hardware during simulation.



Ros2_control hardware interface

ros2_control is the hardware interface, primarily the software layer that enacts actual Unitree hardware, allowing us to simulate the Unitree without actually having one.

- Using ros2_control, we can implement a controller regardless of whether we have a Unitree robot or not.

Controllers

Using a .yaml file, we can create a program to launch a controller in `ros2_control`, defining what controllers can do (such as what joints they control, the speed, modes of operation, etc.)

```
home > mario > g1_ws > src > g1pilot > config > / g1_controllers.yaml

1 controller_manager:
2   ros_parameters:
3     update_rate: 500 # Hz
4
5   joint_state_broadcaster:
6     type: joint_state_broadcaster/JointStateBroadcaster
7
8   whole_body_controller:
9     type: joint_trajectory_controller/JointTrajectoryController
10    joints:
11      - left_hip_pitch_joint
12      - left_hip_roll_joint
13      - left_hip_yaw_joint
14      - left_knee_joint
15      - left_ankle_pitch_joint
16      - left_ankle_roll_joint
17      - right_hip_pitch_joint
18      - right_hip_roll_joint
19      - right_hip_yaw_joint
20      - right_knee_joint
21      - right_ankle_pitch_joint
22      - right_ankle_roll_joint
23      - waist_yaw_joint
24      - waist_roll_joint
25      - waist_pitch_joint
26      - left_shoulder_pitch_joint
27      - left_shoulder_roll_joint
28      - left_shoulder_yaw_joint
29      - left_elbow_joint
30      - left_wrist_roll_joint
31      - left_wrist_pitch_joint
32      - left_wrist_yaw_joint
33      - right_shoulder_pitch_joint
34      - right_shoulder_roll_joint
35      - right_shoulder_yaw_joint
36      - right_elbow_joint
37      - right_wrist_roll_joint
38      - right_wrist_pitch_joint
39      - right_wrist_yaw_joint
40
41    command_interfaces:
42      - position
43
44    state_interfaces:
45      - position
46      - velocity
47
48    state_publish_rate: 100.0
49    action_monitor_rate: 20.0
50    allow_partial_joints_goal: false
51
52    constraints:
53      goal_time: 2.0
54      stopped_velocity_tolerance: 0.01
55
56
```

Gazebo Launch File

Launch files are used to start and open the simulation environment. We will be able to view the robot model, its physics, ROS 2 control, visualization, etc.

```
home > mario > git_ws > src > gipilot > launch >  g1_gazebo.launch.py

1  from launch import LaunchDescription
2  from launch.actions import IncludeLaunchDescription
3  from launch.launch_description_sources import PythonLaunchDescriptionSource
4  from launch_ros.actions import Node
5  from ament_index_python.packages import get_package_share_directory
6  import os
7
8
9  def generate_launch_description():
10
11     pkg_share = get_package_share_directory('gipilot')
12
13     # Load robot description (URDF)
14     robot_description_path = os.path.join(
15         pkg_share,
16         'description_files',
17         'urdf',
18         '29dof.urdf',
19     )
20     with open(robot_description_path, 'r') as infp:
21         robot_description = infp.read()
22
23     # Gazebo
24     gazebo = IncludeLaunchDescription(
25         PythonLaunchDescriptionSource([
26             os.path.join(
27                 get_package_share_directory('ros_gz_sim'),
28                 'launch',
29                 'gz_sim.launch.py'
30             ),
31         ]),
32         launch_arguments={('gz_args': '-r empty.sdf').items()}
33     )
34
35     # Robot State Publisher (uses robot_description param)
36     robot_state_publisher = Node(
37         package='robot_state_publisher',
38         executable='robot_state_publisher',
39         name='robot_state_publisher',
40         output='screen',
41         parameters=[('robot_description': robot_description)],
42     )
43
44     # Spawn robot in Gazebo from URDF string
45     spawn_entity = Node(
46         package='ros_gz_sim',
47         executable='create',
48         arguments=[
49             '-string', robot_description,
50             '-name', 'g1',
51             '-allow_renaming', 'true'
52         ],
53         output='screen'
54     )
55
56     # Controller manager parameters
57     controller_params = os.path.join(
58         pkg_share,
59         'config',
60         'g1_controllers.yaml'
61     )
62
63     # Load controller manager with same robot_description param
64     controller_manager = Node(
65         package='controller_manager',
66         executable='ros2_control_node',
67         parameters=[('robot_description': robot_description), controller_params],
68         output='both'
69     )
70
71     return LaunchDescription([
72         gazebo,
73         robot_state_publisher,
74         controller_manager,
75         spawn_entity,
76     ])
77
78
```

Creating our own simulations files

Necessary packages for creating our own launch.py file that allows us to simulate the G1 robot are from ROS2's:

- robot_state_publisher
- joint_state_publisher_gui
- teleop_twist_keyboard

We must create a controller node (similar to the one from g1pilot's teleoperation file, but without the need for a physical controller.

- Now must create a launch file implementing both Rviz (visualization) and Gazebo (physics) as well for physics implementation.

Unitree's Github

- Downloading the unitree_ros package, we obtain the g1_description file containing the .urdf (for 29 degrees of freedom) and the meshes.
- Next steps will be to make the controller (for walking) and a joint state broadcaster (ensuring the robot is standing).
- The controller for walking must ensure Trajectory Control is implemented. A simple position controller is not enough due to the complicated physics for walking. Two programs therefore must be made: a joint state broadcaster and joint trajectory controller.
 - Joint state broadcaster: publishes joint states
 - Joint trajectory controller: controls all 29 joints in a coordinated fashion, ensuring balance.

G1 Joint Trajectory Controller

For now am making controller focus on solely walking (no upper body joints needed).

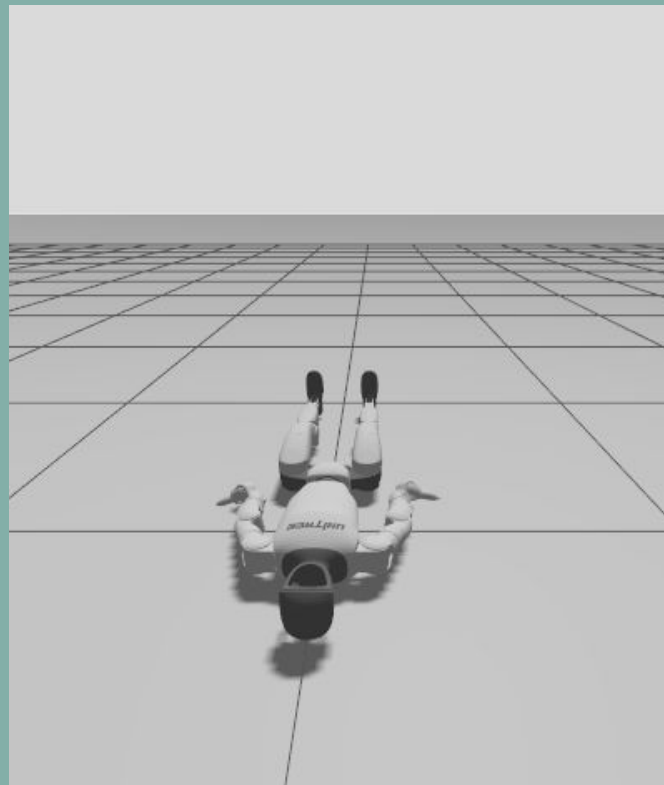
- Waist (3 D.O.F)
- Left Leg + Right Leg (6 D.O.F each)

Progress as of 12/22/2025

- Obtained a 29 D.O.F urdf file for the G1 from the unitree_ros github.
 - Implemented ros2_control code onto the urdf to use ros2_control.
 - Implemented plugin for ros2_control
- Obtained mesh files from the unitree_ros git file
- Created a .yaml file representing the trajectory controller
- Created a launch file to launch Gazebo with the set of files.
-

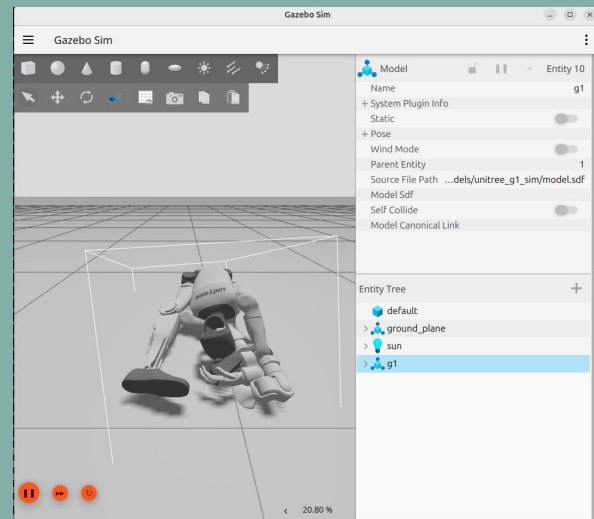
Progress as of 12/22/2025 (continued)

- At the moment, we spawn the Unitree G1 robot onto Gazebo, although it instantly falls into the ground.
 - This tells us that the controller is not yet function (Currently attempting to fix this as it should then be able to simulate walking)
- Obtaining various errors regarding `ros2_control` plugins which may be why the Unitree robot is not capable of standing or walking yet.



Progress as of 12/29/2025

- Have been able to remove almost all the errors. An error regarding the controller remains and am currently trying to find out how to fix this.
- Because of this, the robot still immediately falls into the floor right after it spawns.



```
mario@mario-Inspiron-15-3511:~/g1_ws$ ros2 launch unitree_g1_sim g1_gazebo.launch.py
[INFO] [launch]: All log files can be found below /home/mario/.ros/log/2025-12-29-16-57-00-907607-mario-Inspiron-15-3511-22815
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [gazebo-1]: process started with pid [22818]
[INFO] [robot_state_publisher-2]: process started with pid [22819]
[INFO] [create-3]: process started with pid [22820]
[create-3] [INFO] [1767045421.301355831] [ros_gz_sim]: Requesting list of world names.
[robot_state_publisher-2] [WARN] [1767045421.304091712] [kdl_parser]: The root link pelvis has an inertia specified in the URDF, but KDL does not support a root link with an inertia.
As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-2] [INFO] [1767045421.304407485] [robot_state_publisher]: Robot initialized
[create-3] [INFO] [1767045421.962490268] [ros_gz_sim]: Entity creation successful.
[INFO] [create-3]: process has finished cleanly [pid 22820]
[gazebo-1] [ERROR] [1767045423.688264269] [GazeboSimROS2ControlPlugin]: Gazebo ros2 control found an empty parameters file. Failed to initialize.
```

Progress as of 12/31/2025

- Officially have debugged and fixed all errors from running “ros2 launch unitree_g1_sim g1_gazebo.launch.py”
- In addition, have fixed errors regarding the controller manager not running.
- Next steps: utilize the controller manager to walk the G1 robot.

Startup

Note the startup code for Gazebo:

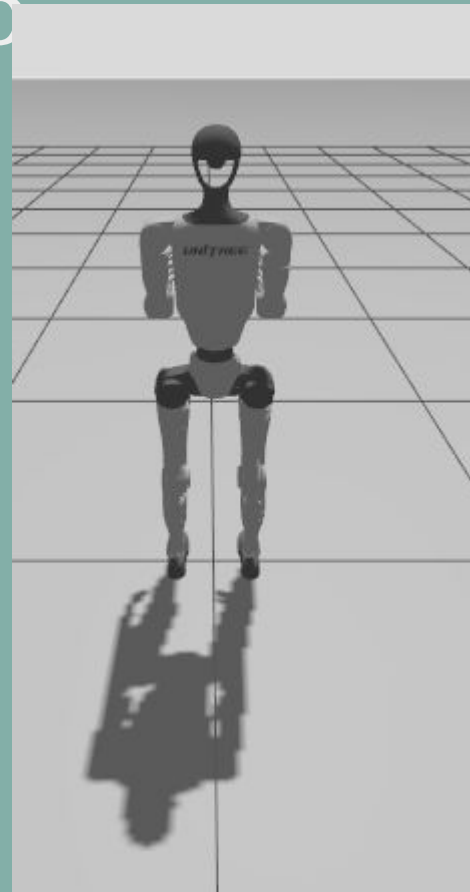
```
source /opt/ros/jazzy/setup.bash
source ~/g1_ws/install/setup.bash
export
GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:$HOME/g1_ws/install/unitree_g1_sim/share/unitree_g1_sim/models
export
GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:$HOME/g1_ws/install/unitree_g1_sim/share/unitree_g1_sim/models
```

Progress as of 01/08/2026

Although I have been able to spawn the G1 Unitree robot on Gazebo, I have not been able to get a controller to have the robot walk properly.

Progress as of 01/30/2026

- Instead of taking Gazebo libraries from ROS2, I installed Gazebo into the directory that launches the g1 robot.
- Now running the launch file, the G1 robot stands up and remains standing, signifying some physics is working.
- The only thing now that is not working is the controllers “g1_controllers.yaml”. Because of that, there is still no walking capabilities.
- From debugging, the main issue may stem from the hardware interface not understanding the .yaml file.



```

mario@mario-Inspiron-15-3511:~/g1_ws$ ros2 launch unitree_g1_sim g1_gazebo.launch.py
[INFO] [launch]: All log files can be found below /home/mario/.ros/log/2026-01-30-15-45-22-415331-mario-Inspiron-15-3511-86788
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [gazebo-1]: process started with pid [86791]
[INFO] [robot_state_publisher-2]: process started with pid [86792]
[INFO] [create-3]: process started with pid [86794]
[INFO] [spawner-4]: process started with pid [86795]
[INFO] [spawner-5]: process started with pid [86796]
[create-3] [INFO] [1769805922.835975621] [ros_gz_sim]: Requesting list of world names.
[robot_state_publisher-2] [WARN] [1769805922.841396734] [kdl_parser]: The root link pelvis has an inertia specified in the URDF, but KDL does not support a root link with an inertia.
As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-2] [INFO] [1769805922.841676221] [g1.robot_state_publisher]: Robot initialized
[spawner-4] [INFO] [1769805923.221071553] [spawner_joint_state_broadcaster]: waiting for service /g1/controller_manager/list_controllers to become available...
[create-3] [INFO] [1769805923.505063632] [ros_gz_sim]: Entity creation successful.
[INFO] [create-3]: process has finished cleanly [pid 86794]
[gazebo-1] [INFO] [1769805925.213708285] [g1.gz_ros_control]: Loading controller_manager
[gazebo-1] [INFO] [1769805925.227923370] [g1.controller_manager]: Using ROS clock for triggering controller manager cycles.
[gazebo-1] [INFO] [1769805925.231706849] [g1.controller_manager]: Subscribing to '/g1/robot_description' topic for robot description.
[gazebo-1] [WARN] [1769805925.234249714] [g1.gz_ros_control]: Waiting RM to load and initialize hardware...
[gazebo-1] [INFO] [1769805925.652427569] [g1.controller_manager]: Received robot description from topic.
[gazebo-1] [INFO] [1769805925.662459566] [g1.gz_ros_control]: The position_proportional_gain has been set to: 0.1
[gazebo-1] [INFO] [1769805925.662903358] [g1.gz_ros_control]: Loading joint: waist_yaw_joint
[gazebo-1] [INFO] [1769805925.662928019] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.662938201] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663031452] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.663170509] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.663177015] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663431343] [g1.gz_ros_control]: Loading joint: waist_roll_joint
[gazebo-1] [INFO] [1769805925.663442108] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.663446785] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663457609] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.663462917] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.663473927] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663568774] [g1.gz_ros_control]: Loading joint: waist_pitch_joint
[gazebo-1] [INFO] [1769805925.663578988] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.663583851] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663593642] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.663598583] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.663602626] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663635138] [g1.gz_ros_control]: Loading joint: left_hip_yaw_joint
[gazebo-1] [INFO] [1769805925.663641565] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.663645471] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.663650471] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.663655318] [g1.gz_ros_control]: Command:

```

Progress as of 01/30/2026

```
[gazebo-1] [INFO] [1769805925.664803193] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.664806859] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664822116] [g1.gz_ros_control]: Loading joint: right_wrist_roll_joint
[gazebo-1] [INFO] [1769805925.664827566] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.664831318] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664835682] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.664840045] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.664844161] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664859324] [g1.gz_ros_control]: Loading joint: right_wrist_pitch_joint
[gazebo-1] [INFO] [1769805925.664864494] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.664868362] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664873247] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.664877443] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.664880907] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664897950] [g1.gz_ros_control]: Loading joint: right_wrist_yaw_joint
[gazebo-1] [INFO] [1769805925.664903337] [g1.gz_ros_control]: State:
[gazebo-1] [INFO] [1769805925.664906978] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.664911471] [g1.gz_ros_control]: velocity
[gazebo-1] [INFO] [1769805925.664915775] [g1.gz_ros_control]: Command:
[gazebo-1] [INFO] [1769805925.664919783] [g1.gz_ros_control]: position
[gazebo-1] [INFO] [1769805925.665874334] [g1.controller_manager]: Initialize hardware 'G1System'
[gazebo-1] [WARN] [1769805925.665929698] [g1.controller_manager]: Executor is not available during hardware component initialization for 'G1System'. Skipping node creation!
[gazebo-1] [INFO] [1769805925.666423218] [g1.controller_manager]: Successful initialization of hardware 'G1System'
[gazebo-1] [INFO] [1769805925.667047516] [resource_manager]: 'configure' hardware 'G1System'
[gazebo-1] [INFO] [1769805925.667056990] [g1.gz_ros_control]: System Successfully configured!
[gazebo-1] [INFO] [1769805925.667066410] [resource_manager]: Successful 'configure' of hardware 'G1System'
[gazebo-1] [INFO] [1769805925.667080243] [resource_manager]: 'activate' hardware 'G1System'
[gazebo-1] [INFO] [1769805925.667086766] [resource_manager]: Successful 'activate' of hardware 'G1System'
[gazebo-1] [WARN] [1769805925.667155925] [g1.controller_manager]: Component 'G1System' does not have read or write statistics initialized, skipping registration.
[gazebo-1] [INFO] [1769805925.667165729] [g1.controller_manager]: Resource Manager has been successfully initialized. Starting Controller Manager services...
[gazebo-1] [ERROR] [1769805925.724821381] [g1.controller_manager]: The 'type' param was not defined for 'joint_state_broadcaster'.
[spawner-4] [FATAL] [1769805925.726532523] [spawner_joint_state_broadcaster]: Failed loading controller joint_state_broadcaster
[spawner-5] [INFO] [1769805925.852240819] [spawner_g1_controller]: waiting for service /g1/controller_manager/list_controllers to become available...
[ERROR] [spawner-4]: process has died [pid 86795, exit code 1, cmd '/opt/ros/jazzy/lib/controller_manager/spawner joint_state_broadcaster --controller-manager /g1/controller_manager --ros-args'].
[gazebo-1] [ERROR] [1769805926.103930477] [g1.controller_manager]: The 'type' param was not defined for 'g1_controller'.
[spawner-5] [FATAL] [1769805926.105202333] [spawner_g1_controller]: Failed loading controller g1_controller
[ERROR] [spawner-5]: process has died [pid 86796, exit code 1, cmd '/opt/ros/jazzy/lib/controller_manager/spawner g1_controller --controller-manager /g1/controller_manager --ros-args'].
[gazebo-1] [WARN] [1769805927.584633989] [g1.gz_ros_control]: Desired controller update period (0.01 s) is slower than the gazebo simulation period (0.001 s).
[gazebo-1] [WARN] [1769805927.595516333] [g1.controller_manager]: No clock received, using time argument instead! Check your node's clock configuration (use_sim_time parameter) and if a valid clock source is available
[gazebo-1] [WARN] [1769805928.607758158] [g1.controller_manager]: No clock received, using time argument instead! Check your node's clock configuration (use_sim_time parameter) and if
```

What to do next

- Looking more into yaml files in general.
- Going to look into what I can do on Gazebo without the controller.
- Spend more time studying ROS2 and Gazebo

What is Gazebo:

- Open sourced 3D robotics simulator
- Enables design, testin, and simulation of robots in complex indoor and outdoor environments
- Has high-fidelity physics and sensor modeling
- Integrates with the Robot Operating System (ROS) and uses physics engines like ODE
- Creator: Andrew Howard (now stewarded by Open Robotics, the primary creator and maintainer of open-source software for robotics).